

Backend Deficiencies, Git, and GitHub

PART A - BACKEND DEFICIENCIES

1. What is Meant by Backend Deficiency?

The backend of an application is the server-side portion that handles business logic, database operations, authentication, and communication with the frontend through APIs. A 'backend deficiency' refers to a weakness, flaw, or gap in this server-side system that prevents the application from functioning efficiently, securely, or reliably. These deficiencies may not always be visible to end users immediately, but they tend to surface as performance problems, security breaches, or system failures as the application grows in usage and complexity.

Identifying and addressing backend deficiencies early in development is important because backend issues are often far more expensive and disruptive to fix once an application is in production, since they can affect data integrity, user trust, and the stability of the entire system.

1.1 Common Types of Backend Deficiencies

Deficiency	Description	Typical Impact
Poor Scalability	The system is built in a way that cannot handle increasing users, data, or traffic	Slow response times or crashes during high load
Weak Security Practices	Missing input validation, weak authentication, or unencrypted data	Vulnerability to attacks such as SQL injection or data breaches
Inefficient Database Design	Poorly structured tables, missing indexes, or redundant data	Slow queries and increased server load
Lack of Error Handling	Errors are not caught or logged properly	Application crashes with no useful information for debugging
Tight Coupling	Components depend heavily on each other, making changes risky	Difficult to update or scale individual parts of the system
Insufficient Testing	Few or no automated tests covering backend logic	Bugs go unnoticed until they affect real users
Poor Documentation	APIs and code are not clearly documented	Slower onboarding and increased risk of miscommunication in teams

1.2 Addressing Backend Deficiencies

- **Design for scale early:** Use load balancing, caching, and efficient database indexing so the system can grow without major rework.
- **Follow secure coding standards:** Validate all inputs, use parameterized queries, hash passwords, and apply authentication and authorization consistently.
- **Implement structured logging and error handling:** Ensure failures are caught, logged, and reported in a way that helps diagnose issues quickly.
- **Adopt modular architecture:** Reduce tight coupling by separating concerns into independent services or layers (e.g., controllers, services, data access).

- **Write automated tests:** Unit and integration tests catch regressions before they reach production.
- **Maintain clear documentation:** Document APIs, configurations, and architecture so the system remains maintainable as teams change over time.

PART B - GIT AND GITHUB

2. What is Git?

Git is a free, open-source distributed version control system used to track changes in source code during software development. It allows multiple developers to work on the same project simultaneously without overwriting each other's changes, and it keeps a complete history of every modification ever made to the project. Because Git is distributed, every developer has a full copy of the project's history on their own machine, which means most operations can be performed offline and the project remains safe even if a central server is unavailable.

2.1 Core Git Concepts

Term	Meaning
Repository (Repo)	A folder tracked by Git that contains the project's files along with their complete history of changes.
Commit	A saved snapshot of changes made to the files, along with a message describing what was changed and why.
Branch	An independent line of development, allowing new features or fixes to be worked on without affecting the main codebase.
Merge	The process of combining changes from one branch into another.
Clone	Creating a local copy of a remote repository on your own machine.
Pull / Push	Pull downloads changes from a remote repository; push uploads local changes to a remote repository.

2.2 Why Git is Used

- **Version history:** Every change is recorded, so previous versions of the code can always be restored if something breaks.
- **Collaboration:** Multiple developers can work on different features in parallel using branches, then merge their work together safely.
- **Backup and safety:** Since the full history is distributed, the project is protected against data loss from a single point of failure.
- **Accountability:** Commits are attributed to specific authors, making it clear who made which change and when.

3. What is GitHub?

GitHub is a cloud-based hosting platform built around Git, allowing developers to store, manage, and collaborate on Git repositories online. While Git is the underlying version control technology, GitHub adds a web-based interface and a range of collaboration features on top of it, making it easier for individuals and teams to manage projects, review code, and coordinate work, especially in remote or distributed teams.

3.1 Key Features of GitHub

Feature	Purpose
Remote Repository Hosting	Stores a central, shared copy of the project that all team members can push to and pull from.
Pull Requests	A formal way to propose, review, discuss, and merge changes from one branch into another before they become part of the main codebase.
Issues	A built-in tracker for reporting bugs, requesting features, and managing tasks.
GitHub Actions	Automation tool used to run tests, build, and deploy code automatically whenever changes are made (CI/CD).
Code Review Tools	Allows team members to comment on specific lines of code, suggest changes, and approve or request modifications.
Access Control	Manages who can view, edit, or administer a repository, supporting both public and private projects.

3.2 Git vs. GitHub

Aspect	Git	GitHub
What it is	A version control system (software/tool)	A cloud platform that hosts Git repositories
Where it runs	Runs locally on a developer's machine	Runs on remote servers, accessed via the web
Core function	Tracks changes and history of files	Provides hosting, collaboration, and project management on top of Git
Internet required?	No, works fully offline	Yes, for syncing, collaboration, and hosting

Note: Git is the underlying technology that manages versions of code, while GitHub is one of several platforms (others include GitLab and Bitbucket) that host Git repositories and add collaboration features such as pull requests, issue tracking, and automated workflows.